



OPTIMA

AUTOMATED STUDY FLOW

TESTING REPORT

Julian C · Year 12 Software Engineering
Mentor: Ms West · Client: Sachin Kanagasundaram
Term 2, 2026 · v1.0 · 29 May 2026

Testing Report

Developer-side test plan, the strategies used to verify each part of the application, the code-optimisation techniques applied, and the outcomes observed across the three prototype releases and the final v1.0 build.

CONTENTS

- 1. Testing approach
- 2. Types of testing planned and performed
- 3. Developer-side test plan
- 4. Test results by feature
- 5. Defect log and resolutions
- 6. Code optimisation techniques
- 7. Performance measurements
- 8. Regression checklist

1. Testing approach

Optima is small enough that an automated test framework would have cost more than it returned. Instead, testing is structured around **three feedback loops**, each used at a different point in the cycle:

Loop	When	Cost / sec	Catches
Inner → manual smoke test	After every meaningful change.	~90 secs	Crashes, broken routes, regressions in the golden path.
Middle → HTTP route check	Before each prototype release	~5 secs scripted	Typos, missing handlers, 500 errors, redirects pointing to the wrong place.
Outer → Evaluator round	Once per prototype.	2-3 days wall time.	UX issues, security issues, real-data bugs – things the developer might be blind to.

2. Types of testing planned and performed.

Type	Purpose	Performance in Optima	Verdict
Unit-level checks	Verify that individual pure functions produce correct values	Manual checks in python3 -i (e.g. priority_score(a, today) vs four hand computed cases (see §4.3)	PASSED ▾
Module / Integration	Check that the 2 modules are communicating correctly with each other.	Store a user via storage.save_user, reload via storage.load_user and assert all fields round-trip.	PASSED ▾
System / end-to-end	Follow the whole user journey from signup to schedule.	Route smoke test (using the Flask test client) (tools/smoke_test.py): all 13 GET routes return 200/303/404, never 500.	PASSED ▾
Acceptance / user	Confirm that the application solves the real problem of the client.	3 prototype rounds with 3 end-users + client (see Evaluations)	PASSED ▾
Boundary	Probe the edges of valid input.	Username at 2, 3, 32, 33 chars; password at 5 and 6 chars; due date today vs yesterday; weighting 0, 100, 101.	PASSED ▾
Negative	Confirm rejected inputs are rejected cleanly.	Log in with an unknown user, log in with an empty password, sign up with a non-matching confirmation, and constrain with end_time < start_time.	PASSED ▾
Security	Verify that the security claims hold up.	Two users with the same password produce different hashes; protected routes return 401 when unauthenticated; sessions clear on logout; the login rate limit triggers after the 5th failed attempt; encrypted private notes round-trip cleanly via the web UI but appear opaque (base64) in the local JSON ; the HMAC tag verification rejects tampered ciphertext.	PASSED ▾
Crash / durability	Confirm an interrupted save can't corrupt the data file.	Kill -9 mid-save was reproduced 3 times. On relaunch, the file is always the prior-committed version.	PASSED ▾
Cross-platform sanity	Catch macOS-only assumptions.	Manual run on macOS 14 (Apple Silicon) and Windows 11 in a VM. The --browser flag was added for cases where pywebview's renderer doesn't initialise.	PASSED ▾
Accessibility	WCAG 2.1 AA, where applicable,	Manual keyboard-only walkthrough; colour-contrast check (default + high-contrast palettes); zoom up	PASSED ▾

	for a single-user desktop app.	to 2.0x with layout integrity.	
Load	Confirm Optima doesn't fall over on a realistic dataset.	Seeded 50 assignments + 49 constraints; server-side dashboard regeneration stays at a few milliseconds, no errors or slowdown.	PASSED ▾

3. Developer-side test plan

The plan below is the structured 'what/why/outcome' record, where each row is a single planned test, the reason it earns its place, and the results when it was run.

3.1 Authentication

Test	Why	Expected	Outcome
Sign up with a username 3 chars long.	Boundary of validate_username	200 OK, account created	PASSED ▾
Sign up with a username 2 chars long.	Boundary → one below the floor.	The form was re-rendered with a typed error.	PASSED ▾
Sign up with a username containing a space.	Pattern [A-Za-z0-9.+\\-@]	Rejected	PASSED ▾
Sign up with a non-matching confirm.	Cross-field validation.	Rejected with 'passwords don't match.'	PASSED ▾
Login with the wrong password for a valid user.	Authentication must not say which half was wrong.	Generic 'Invalid username or password'	PASSED ▾
Login with an unknown username.	No enumeration (same as the previous test)	Generic message.	PASSED ▾
Two users, the same passwords, and different salts.	Confirms per-user salt is real.	Different password_hash strings in the .json.	PASSED ▾
Access /dashboard without login.	Session-based access control.	Redirect to /login.	PASSED ▾
Logout clears the session.	Revisiting the protected route after login redirects.	Redirect to /login.	PASSED ▾

3.2 Persistence & data integrity

Test	Why	Expected	Outcome
Save a User, then reload; every field is equal.	Confirms the to_dict / from_dict contract.	Field-by-field equality	PASSED ▾

kill -9 during save and relaunch	Atomic-write claim	Old file intact, no truncated JSON.	PASSED ▾
Two accounts (test1, test2): test1 cannot see test2's data.	Isolation between accounts.	Session-bound; correct user.json is loaded.	PASSED ▾
Delete data/users/julianc.json	Tolerance to file disappearance.	Next save recreates, no crash.	PASSED ▾
Manually corrupt the JSON, then try to log in.	Resilience to file damage.	Friendly error page, no stack trace.	PASSED ▾

3.3 Priority calculator

Four hand-computed cases verify both the formula and urgency override.

Assignment	Weighting	Days left	Expected score	Computed
Maths test	25	2	999 (critical)	999 PASSED ▾
Physics depth study	40	14	28.57	28.57 PASSED ▾
English Essay	40	21	19.05	19.05 PASSED ▾
Modern History Quiz	10	5		20.0 PASSED ▾

Tie-break test: Two assignments with weighting 30, both due in 10 days → same score 30.0 → ordering falls back to due_date. Verified by adding two tasks with due dates differing by one day. The earlier-due task ranks first. PASSED ▾

3.4 Scheduling engine

Test	Why	Expected	Outcome
Schedule one task with 6 h work, no constraints.	Baseline placement.	Three blocks of 2h each, with breaks.	PASSED ▾
The schedule respects class periods on day_of_fornight 0.	Conflict resolver, basic case.	No block overlaps a class.	PASSED ▾
The schedule respects previously placed blocks.	v0.2 overlap bug.	No block overlaps a class.	PASSED ▾
Awake window honoured. (07:00-22:00)	No 03:00 study sessions, no invisible tasks.	All blocks have start_time ≥ 7.0 and end_time ≤ 22.0.	PASSED ▾
A free slot < 30 minutes is not used.	MIN_SESSION threshold.	Slot skipped.	PASSED ▾
Task with more hours than time available.	Graceful partial scheduling.	Schedule what fits; surface the warning.	PASSED ▾

3.5 GUI

Test	Why	Expected	Outcome
Every link in the sidebar reaches its target.	Catches broken nav.	200 OK on each.	PASSED ▾
Theme toggle persists across logout/login.	Preference round-trip.	Same theme after reload.	PASSED ▾
High-contrast mode is legible in glare (visual check)	Accessibility.	The client confirms it's readable outdoors.	PASSED ▾
Zoom from 1.0 doesn't break the grid.	Layout integrity at high zoom.	The weekly grid is still aligned	PASSED ▾
Keyboard-only navigation reaches every interactive element.	Accessibility.	Tab order is sensible, and focus is visible when toggled.	PASSED ▾

4. Test results by feature

Aggregated by feature area across the lifetime of the project.

Feature	Tests planned	Tests run	Passed at v1.0
Authentication	9	9	9/9
Persistence & data integrity	5	5	5/5
Priority calculator	5	5	5/5
Scheduling engine	6	6	6/6
GUI & navigation	5	5	5/5
Preferences & accessibility	5	5	5/5
Subject & constraint CRUD	4	4	4/4
Assignment CRUD	5	5	5/5
Total	44	44	44/44

5. Defect log and resolutions

Every meaningful defect surfaced during testing (developer-side or evaluator-side), what fixed it, and how the fix is verified in the regression checklist (§9).

#	Defect	Surfaced by	Root Cause	Fix	Verified by
D-00	Hashing is one-way, whereas the rubric states both encryption and decryption.	Post-build audit through rubric.	SHA-256 alone doesn't satisfy the decryption line.	Added app/crypto.py, PBKDF2-HMAC-SHA-256 key derivation + SHA-256-CTR stream cipher + HMAC-SHA-256 authentication tag for the per-assignment private notes field.	python -m app.crypto self-test. Round trip verified through the UI.
D-01	Two same-day study blocks overlap on the weekly grid.	Client (Round 2)	The greedy cursor didn't advance past the previously placed block.	Added find_free_slots() that subtracts existing blocks from the awake window before placement.	Test 3.4, row 3 (regression)
D-02	Force-quitting mid-save corrupts the user JSON	E3 (Round 2)	Naive open(p,'w') can leave a half-written file.	tempfile.mkstemp + fsync + os.replace.	Test 3.2, row 2 (regression)
D-03	Single global salt rainbow table risk.	E3 (Round 2)	Constant SALT in auth.py.	Per-account secrets.token_hex(16) are generated at signup and stored on User.	Test 3.1 row 7 (regression).
D-04	'Critical' red is indistinguishable from regular red.	E2 (Round 1)	Both used the same hex code.	Reserved a distinct, deeper red for the critical-state badge.	Visual check - sidebar legend.
D-05	Settings sidebar icon dead.	E2 (Round 2)	Route stub: no handler exists.	Wired to /preferences.	Test 3.5 row 1 (regression)
D-06	The cushion doughnut is not redrawing after the task edit.	Self (Round 3 polish)	State calculation is cached on the user object.	Recalculate in the route handler on every dashboard load.	Manual test – edit task, observe gauge change.
D-07	The focus outline is too thin (1px) for keyboard navigation.	E3 (Round 3)	Browser default.	Added body.focus-on:focus-visible rule with 3px outline; toggle on preferences.	Test 3.5 row 5 (regression)
D-08	Jinja syntax error in weekly.html (today comparison)	Self (Round 3 polish)	Compared d.isoformat() with a string instead of comparing dates.	Pass today=date.today() into the render context, compare	Test 3.5 row 1 (route returns 200).

directly.

6. Code optimisation techniques

The rubric states that a 'language-dependent code optimisation technique' is a task requirement. Below are specific optimisations applied to Optima, why they matter, and the measured (or estimated) impact it has had.

6.1 Algorithmic – greedy scheduling instead of an exhaustive search

The 'true' study block placement problem is NP-hard. Solving it exactly for a 14-day window with 8-15 assignments would mean exploring millions of permutations. Instead, the scheduler is greedy. It works by ranking tasks once, walking forward once, and placing them into the first slot that fits. $O(n \cdot m)$, where n is the assignment count and m is the free-slot count.

Trade-off: the result may not be globally optimal. The algorithm could 'starve' a later task by greedily filling slots with the earlier one. This is acceptable because:

- + The priority calculator already ranks more urgent tasks first, so the greedy decision is usually the right one.
- + If the cushion gauge goes red, the user has the information they already need to adjust manually.
- + A truly optimal scheduler would block the UI for seconds on every dashboard load.

Measured: a full fortnight schedule for a realistic ~37-event timetable runs in ~9 ms (of which `find_free_slots()` is ~3 ms); an 18x stress test of 666 events still completes under 100 ms. Reproducible via `tools/benchmark.py`; see §7.

6.2 Data structure – sorted FreeSlot intervals

`find_free_slots()` returns a list of `FreeSlot(date, start, end)` already sorted by start time. The placer then walks the list once. If it weren't sorted, every placement would need a linear scan to find the earliest slot: $O(m^2)$ instead of $O(m)$.

6.3 Persistence – lazy load, dirty save

The User document is loaded once per request from `data/users/<name>.json` via `Storage.load_user()` is kept in memory for the duration of the handler and written back only if mutated. No JSON I/O on routes that don't write. e.g. `/dashboard`, `/weekly`, `/monthly` all read but never saved.

6.4 CSS - custom properties instead of duplicate declarations

Every theme colour originates from a CSS custom property that has been defined on `:root`. Dark theme overrides under `[data-theme="dark"]`; high contrast under `[data-contrast="high"]`. Switching the entire colour palette is a single attribute change on the `<html>` element. This way, no per-component JavaScript code reads or writes colour values, and the same stylesheet is able to serve three palettes.

Before the refactor (v0.2): ~80 colour hex literals scattered through style.css, dark mode would have needed every one duplicated under a .dark selector. **After refactor (v0.3):** ~24 named tokens, adding a fourth palette (e.g. sepia) would now take a 24-line block.

6.5 Template - one base + per-page extension

Every view {% extends "base.html" %}. The base supplies the sidebar, the run-head, the JS bootstrap, and the body-level data attributes. Per-view templates only declare what is unique to that specific view. Saved roughly 200-300 lines of duplicated markup vs. standalone HTML files.

6.6 Python - dataclasses paired to to_dict / from_dict

Hand-rolling JSON serialisation per field would have made the storage layer brittle, and forgetting a field on a save or load is a silent bug. The dataclass round-trip means that there is exactly one place to add a field (the dataclass definition + its two round-trip methods), and an omission anywhere shows up as a missing key at load time. This is correctness optimisation instead of speed optimisation, which is also called out by the rubric under 'code optimisation'.

6.7 Front end - AJAX progress slider instead of a full reload.

The progress slider on each task card uses fetch('/api/tasks/<id>/progress', {method: 'PUT', body: ...}) to save inline. Saving 1% progress doesn't repaint the whole dashboard. The latency budget for the slider release-to-save round trip is less than 50ms, and the observed median is ~12ms locally.

7. Performance measurements

Two reproducible harnesses back these numbers: tools/benchmark.py in the repository (scheduler hot paths, timed with timeit across a range of timetable sizes) and a one-off Flask test client timing for the storage and rendering figures. Measured on Apple Silicon (arm64), Python 3.13. Exact milliseconds vary by machine; run the script to reproduce.

Measurement	Conditions	Time
find_free_slots()	~37-event timetable, 14-day horizon	~3ms
generate_schedule() (full placement)	~37-event timetable, 14-day horizon	~9ms
load_user()	28.9KB account (12 tasks, 49 constraints)	~0.4ms
save_user incl. fsync + atomic replace	Same account.	~2ms
GET /dashboard (serverside, end-to-end)	Lightly-loaded account, Flask test client	~1-2ms

Scaling – scheduler vs. timetable size

From tools/benchmark.py, scaling stored events from a realistic ~37 up to an 18x stress test of ~666:

Stored events	find_free_slots()	generate_schedule()
37 (realistic)	~3ms	~9ms
133	~9ms	~25ms
666 (stress)	~45ms	~90ms

Growth is roughly linear, confirming the $O(n)$ characterisation. Even at 666 events, far beyond any single student's timetable, a full regeneration stays under 100ms, the threshold below which an interaction feels instant. This is evidence behind the decision not to reach for an interval tree (see *Objectives Checklist*, SW D).

Cold start (launching the native window) was not formally benchmarked. This process is dominated by Python interpreter start-up, the Flask import, and pywebview's OS-level window creation rather than by Optima's own code. Informally, the window appears within a couple of seconds of python run.py.

8. Regression checklist

Run end-to-end before any release; check each item as it passes to ensure functionality.

#	Step	Pass criterion
1	Launch via python run.py	The native window opens within 4s, splash visible.
2	The login screen button is functional.	The login screen appears after the user clicks.
3	Click "Create an account"	The signup form was rendered.
4	Submit signup with username "rgrn1" and password "Password_123"	Lands on the dashboard with 5 default subjects.
5	Add a subject "Chemistry" with the colour "#ff8c42"	Chip appears with the new colour.
6	Add a constraint on day 0, 09:15-10:25, subject "English"	Constraints listed under the 14-day timetable.
7	Add an assignment, "Test essay", due in 5 days, weighting 30%, 6 h.	The task appears on the dashboard.
8	Go to Weekly	There are three study blocks visible across this week.
9	Go to Monthly	The due date dot is visible on the correct day.
10	Drag the progress slider to 50%	Cushion gauge updates on dashboard reload.

Run record: final regression was recorded on Wed 27 May 2026: 20/20 pass.